

# Pattern Classification by Prototype Matching

## Minimun-Distance Classifier

Nuñez Elias - Facundo Sanchez - Tomas Mele - Agustin Frenkel

Universidad de Buenos Aires

Facultad de Ciencias Exactas y Naturales  
Departamento de Computación

Julio 2026

# ¿Qué es el reconocimiento de patrones?

En visión por computadora, un patrón puede entenderse como una disposición espacial de características.

Una clase es un conjunto de patrones que comparten propiedades comunes.

El reconocimiento de patrones busca asignar automáticamente patrones desconocidos a sus respectivas clases.

Ejemplos:

- Reconocimiento facial.
- Clasificación de animales.
- Reconocimiento de caracteres.
- Diagnóstico asistido por imágenes.

# Proceso de reconocimiento

El proceso de reconocimiento suele dividirse en cuatro etapas:

- 1 Detección
- 2 Preprocesamiento
- 3 Extracción de características
- 4 Clasificación



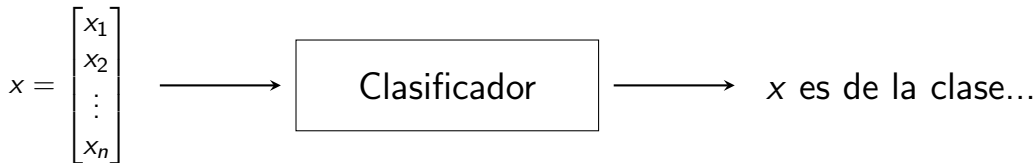
$\Rightarrow$  Preprocesamiento (ej, HoG)  $\Rightarrow$  Clasificador  $\Rightarrow$  Clase = 

Nosotros nos vamos a enfocar en la etapa de **clasificación**.

# ¿Qué es un clasificador?

Podemos pensar un **clasificador** como un algoritmo que reciben unas características  $x$ , de en este caso una imagen, y decide a qué clase pertenece.

Su objetivo es **encontrar similitudes** entre patrones conocidos y patrones desconocidos.

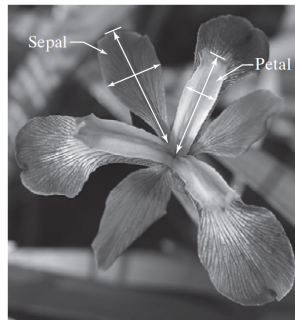


# Extracción de características

Primero transformamos la imagen en representaciones numéricas llamadas características (*features*), generalmente en forma de vector.

Estas características buscan capturar información relevante de la forma, textura o estructura de los objetos presentes en la imagen.

**ATENCIÓN:** La calidad y elección de estas características influye directamente en el desempeño del clasificador.



$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix}$$

$x_1$  = Petal width  
 $x_2$  = Petal length  
 $x_3$  = Sepal width  
 $x_4$  = Sepal length

# Repaso: SIFT

**SIFT (Scale-Invariant Feature Transform)** es un método de extracción de características ampliamente utilizado en visión por computadora.

Su objetivo es detectar puntos distintivos de una imagen y describirlos mediante vectores numéricos.

- Detecta puntos de interés (*keypoints*).
- Genera descriptores robustos.
- Es invariante a escala y rotación.
- Permite comparar imágenes diferentes.

# Repaso: HoG

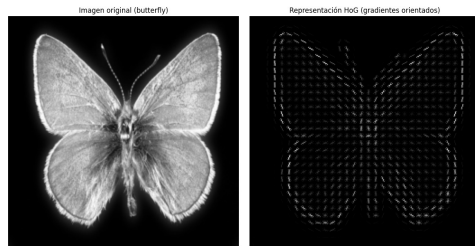
**HoG (Histogram of Oriented Gradients)** es un descriptor basado en bordes y gradientes de intensidad.

En lugar de utilizar directamente los píxeles, resume la distribución de orientaciones de los bordes presentes en una imagen.

Ventajas:

- Captura información de forma y contorno.
- Reduce la sensibilidad a cambios de iluminación.
- Produce vectores de características compactos.

En nuestro código utilizaremos HoG para representar imágenes de animales.



# Métodos de clasificación

Existen tres enfoques principales para la clasificación de patrones:

- ① Clasificación basada en prototipos (*Prototype Matching*)
- ② Clasificadores estadísticos óptimos
- ③ Redes neuronales

En esta exposición nos centraremos **Prototype Matching**



# Patrones y Clases

Un **patrón** es una representación de un objeto o fenómeno mediante un conjunto de características.

Como dijimos antes, representaremos un patrón mediante un vector de características.

Una **clase** es un conjunto de patrones que comparten propiedades comunes.

Por ejemplo, podemos tener varias fotos de animales y separarlas en clases dependiendo el tipo de animal.



# ¿Qué es un prototipo?

Un **prototipo** es una representación de referencia utilizada para describir una clase.

La idea es comparar patrones desconocidos contra estos prototipos y determinar cuál es el más similar.

Dependiendo del problema, una clase puede estar representada por uno o varios prototipos.

Por ejemplo, todas estas imágenes pueden ser usadas para crear un prototipo de la clase **gatos**.



# Pattern Classification by Prototype Matching

**Prototype matching** consiste en comparar un patrón desconocido contra un conjunto de prototipos conocidos.

El patrón se asigna a la clase cuyo prototipo resulte más similar.

La diferencia entre los distintos métodos de matching radica principalmente en la medida de similitud utilizada.

# Minimum-Distance Classifier

## Idea intuitiva

El clasificador puede aplicarse a distintos tipos de elementos de una imagen.  
Por ejemplo, el elemento a clasificar puede ser un píxel, una región o una imagen completa representada como vector.  
Una vez definido qué vamos a clasificar, construimos un prototipo para cada clase.

prototipo = representante típico de una clase

# Minimum-Distance Classifier

Clasificar por cercanía

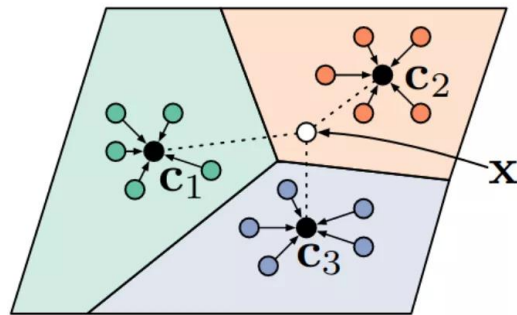
Cuando aparece un nuevo elemento  $x$ ,  
calculamos su distancia a cada prototipo.

$$D_j(x) = \|x - m_j\|$$

Luego buscamos la distancia mínima.

menor distancia  $\Rightarrow$  clase elegida

Es decir, clasificamos  $x$  como perteneciente  
a la clase del prototipo más cercano.



El punto  $x$  se asigna a la región del prototipo  
más cercano.

# Minimum-Distance Classifier

## Definición de los prototipos

Existen distintas formas de definir un prototipo para una clase. En esta demo vamos a utilizar una definición simple: el prototipo de cada clase será su vector medio, es decir, el promedio de los patrones de entrenamiento pertenecientes a esa clase.

$$m_j = \frac{1}{n_j} \sum_{x \in c_j} x, \quad j = 1, 2, \dots, N_c.$$

Aquí,  $m_j$  es el prototipo o vector medio de la clase  $j$ -ésima,  $c_j$  representa la clase  $j$ -ésima,  $n_j$  es la cantidad de patrones utilizados para calcular el promedio de esa clase,  $x$  es un patrón perteneciente a  $c_j$ , y  $N_c$  es el número total de clases.

# Minimum-Distance Classifier

## Distancia a los prototipos

Dado un patrón desconocido  $x$ , el clasificador calcula su distancia a cada prototipo  $m_j$ :

$$D_j(x) = \|x - m_j\|, \quad j = 1, 2, \dots, N_c.$$

Usando distancia euclídea, esto mide qué tan cerca está  $x$  del prototipo de cada clase.

Asignar  $x$  a la clase  $c_i$  si  $D_i(x) < D_j(x)$ ,  $\forall j \neq i$ .

Es decir, el patrón se clasifica en la clase cuyo prototipo se encuentra a menor distancia.

# Minimum-Distance Classifier

Función discriminante: distancia al cuadrado

El clasificador asigna  $x$  a la clase cuyo prototipo está a menor distancia:

$$D_j(x) = \|x - m_j\|.$$

Como la raíz cuadrada no cambia el orden de las distancias, podemos comparar las distancias al cuadrado:

$$D_j^2(x) = \|x - m_j\|^2.$$

Expandiendo:

$$D_j^2(x) = (x - m_j)^T (x - m_j) = x^T x - 2m_j^T x + m_j^T m_j.$$



# Minimum-Distance Classifier

Función discriminante: equivalencia

Para un patrón fijo  $x$ , el término  $x^T x$  es igual para todas las clases, por lo que no afecta la comparación entre prototipos.

Entonces, minimizar  $D_j^2(x)$  es equivalente a maximizar:

$$2m_j^T x - m_j^T m_j.$$

Dividiendo por 2, obtenemos la función discriminante:

$$d_j(x) = m_j^T x - \frac{1}{2} m_j^T m_j.$$

Por lo tanto, elegir la menor distancia es equivalente a elegir el mayor valor de  $d_j(x)$ .

# Minimum-Distance Classifier

## Regla de decisión

El patrón  $x$  se asigna a la clase cuyo valor de  $d_j(x)$  sea mayor:

$$x \in c_i \quad \text{si} \quad d_i(x) > d_j(x), \quad j = 1, 2, \dots, N_c, j \neq i.$$

La frontera de decisión entre dos clases  $c_i$  y  $c_j$  está dada por los puntos donde ambas funciones discriminantes son iguales:

$$d_i(x) = d_j(x).$$

Equivalentemente:

$$d_i(x) - d_j(x) = 0.$$

# Minimum-Distance Classifier

## Frontera de decisión

Usando la expresión de  $d_j(x)$ :

$$d_j(x) = m_j^T x - \frac{1}{2} m_j^T m_j,$$

se obtiene:

$$d_{ij}(x) = d_i(x) - d_j(x)$$

$$d_{ij}(x) = (m_i - m_j)^T x - \frac{1}{2} (m_i - m_j)^T (m_i + m_j) = 0.$$

Esta frontera es el bisector perpendicular del segmento que une los prototipos  $m_i$  y  $m_j$ .

# Correlación

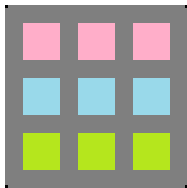
Para buscar prototipos  $w \in \{0, \dots, 255\}^{n \times m}$  conocidos en imágenes nuevas  $f \in \{0, \dots, 255\}^{N \times M}$ , se puede hacer algo simple que es pensar la imagen  $w$  como el prototipo que queremos buscar y usar correlación en cada pixel de la imagen:

$$(w \star f)(x, y) = \sum_s^n \sum_t^m w(s, t) f(x + s, y + t)$$

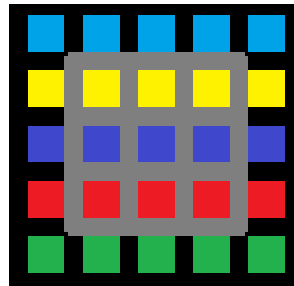
Y esto se hace para cada pixel de la imagen

# Correlación

Si uno piensa las imágenes  $f_{(x,y)}$  y  $w$  como vectores en  $R^{n \cdot m}$ .



$w$



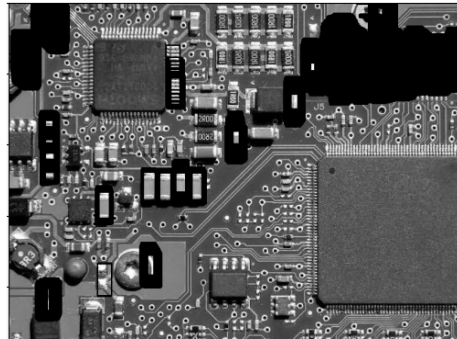
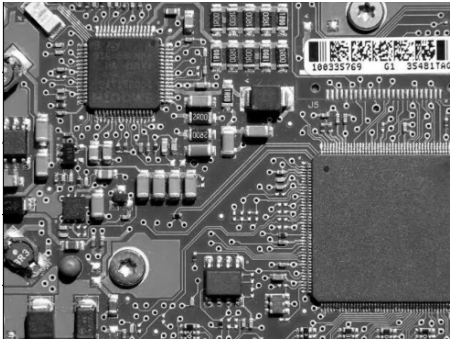
$f$



Esta operación es equivalente al producto escalar entre  $w$  y una parte de  $f$ .

$$(w \star f)(x, y) = \langle w, f_{(x,y)} \rangle = \sum_{i=1}^{n \cdot m} w_i * f_{(x,y)_i} = \|w\| \|f_{(x,y)}\| \cos(\theta)$$

Que mientras mas parecido sean los vectores, el angulo entre ellos seria mas cercano a 0, por lo que se maximizaria cuando el angulo entre  $w$  y  $f$  sea bajo



Ejemplo donde se busca el capacitor en el pcb.

# Correlacion

Busco que solo se pueda maximizar  $\gamma$  con  $\theta$ .

Entonces se usa:

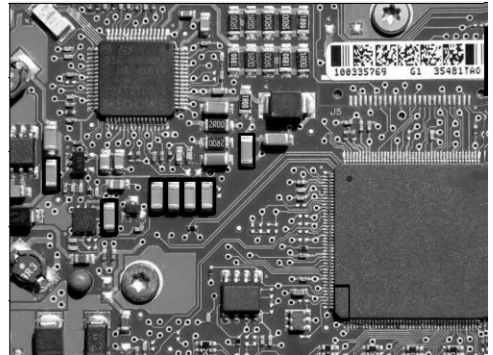
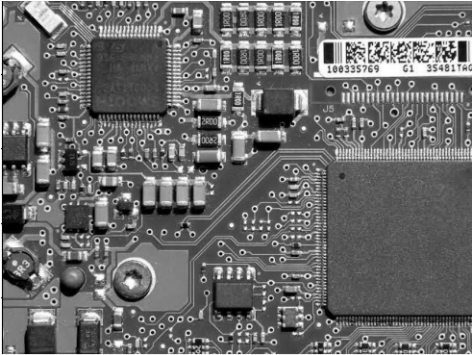
$$\gamma(x, y) = \frac{\sum_s \sum_t [w(s, t) - \bar{w}][f(x + s, y + t) - \bar{f}_{xy}]}{\sqrt{\sum_s \sum_t [w(s, t) - \bar{w}]^2 \sum_s \sum_t [f(x + s, y + t) - \bar{f}_{xy}]^2}} \in [-1, 1]$$

Para ver donde esta w en f:

- puedo buscar los maximos de  $\gamma$
- ver los puntos donde  $\gamma(x, y)$  es mayor que un numero que elijo aparte.



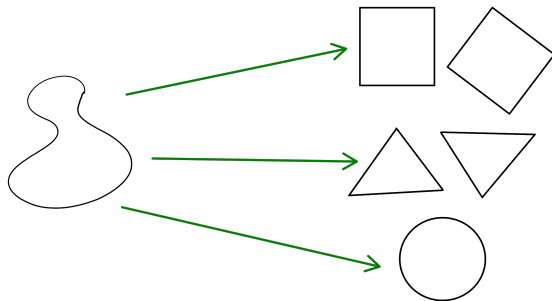
# Correlacion



Mismo ejemplo pero con la otra formula

## Shape Number

# Queremos clasificar siluetas

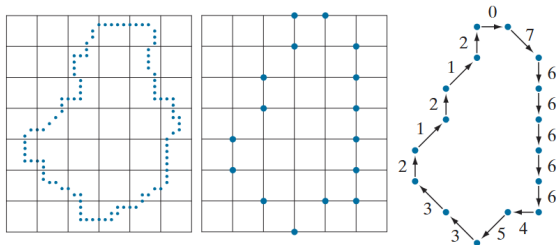


# Shape Number

¿Qué característica significativa podemos extraer de una silueta para compararla con otras?

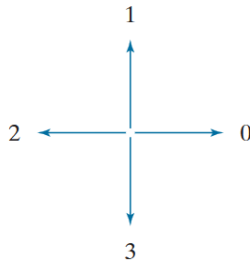
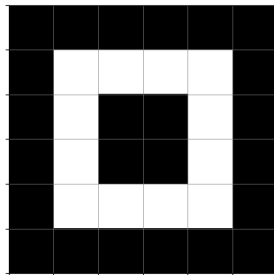
El Shape Number! (número de forma para los de boca :))

Es un código que nos describe el contorno de un objeto de una manera única, permitiéndonos identificar la forma sin importar su tamaño, posición u orientación.



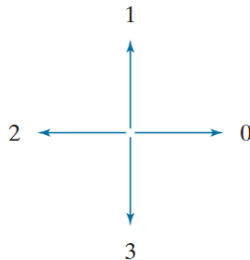
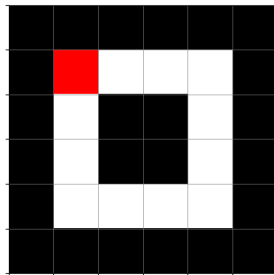
# Shape Number

¿Cómo lo obtengo? Veamos un pequeño ejemplo.



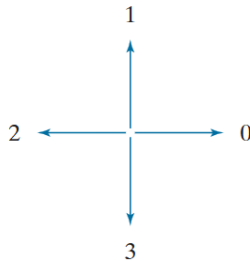
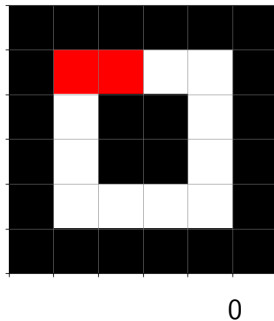
# Shape Number

¿Cómo lo obtengo? Veamos un pequeño ejemplo.



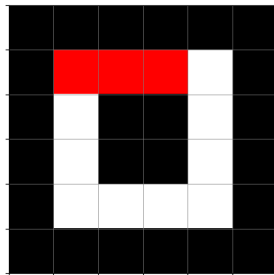
# Shape Number

¿Cómo lo obtengo? Veamos un pequeño ejemplo.

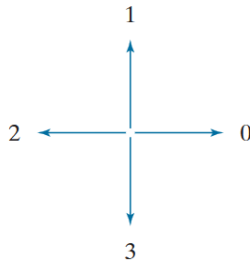


# Shape Number

¿Cómo lo obtengo? Veamos un pequeño ejemplo.

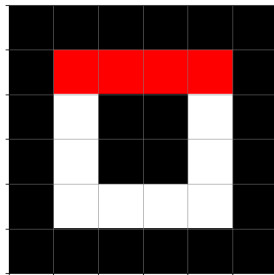


00

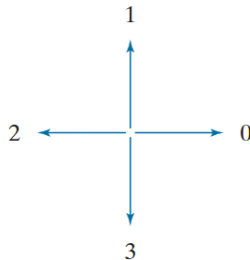


# Shape Number

¿Cómo lo obtengo? Veamos un pequeño ejemplo.



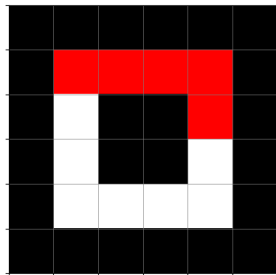
000



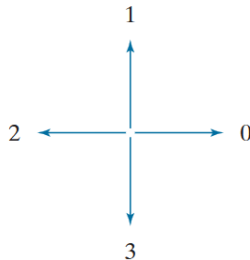


# Shape Number

¿Cómo lo obtengo? Veamos un pequeño ejemplo.

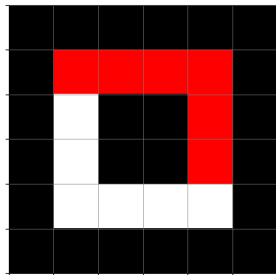


0003

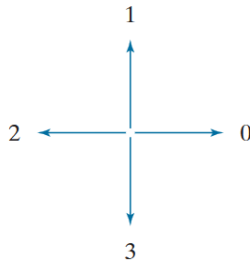


# Shape Number

¿Cómo lo obtengo? Veamos un pequeño ejemplo.

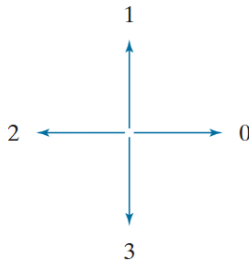
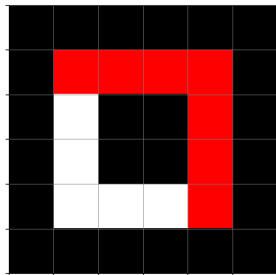


00033



# Shape Number

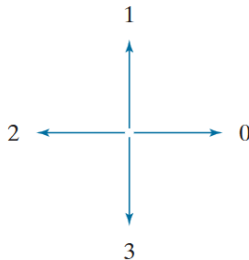
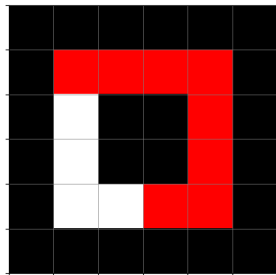
¿Cómo lo obtengo? Veamos un pequeño ejemplo.



000333

# Shape Number

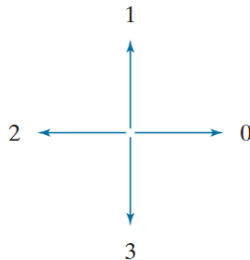
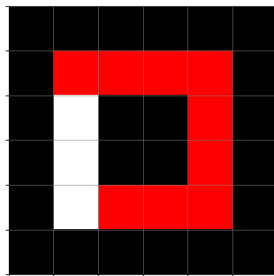
¿Cómo lo obtengo? Veamos un pequeño ejemplo.



0003332

# Shape Number

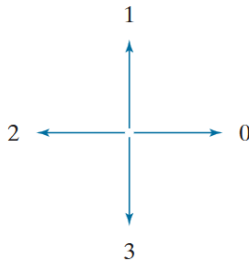
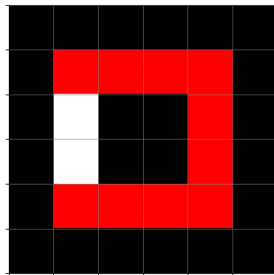
¿Cómo lo obtengo? Veamos un pequeño ejemplo.



00033322

# Shape Number

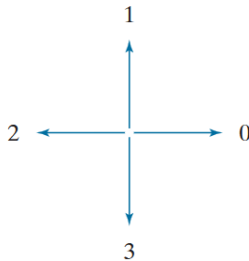
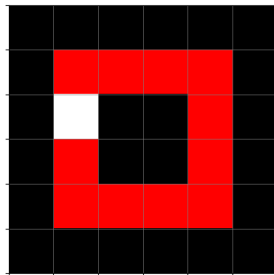
¿Cómo lo obtengo? Veamos un pequeño ejemplo.



000333222

# Shape Number

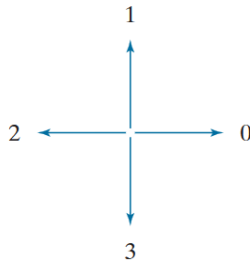
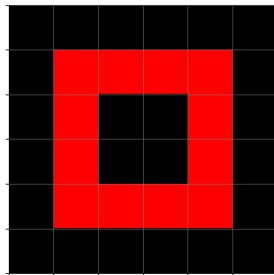
¿Cómo lo obtengo? Veamos un pequeño ejemplo.



0003332221

# Shape Number

¿Cómo lo obtengo? Veamos un pequeño ejemplo.

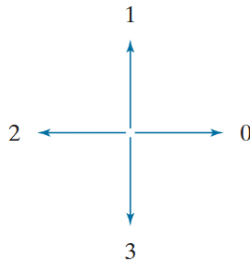
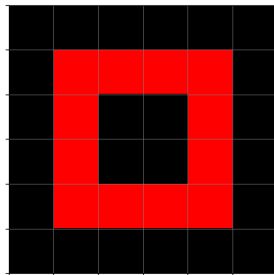


00033322211



# Shape Number

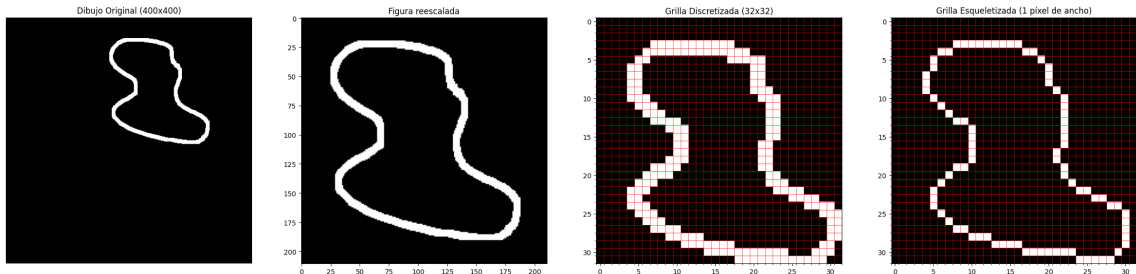
¿Cómo lo obtengo? Veamos un pequeño ejemplo.



000333222111

# Shape Number

Con figuras más complejas se complica un poco la cosa...



## Shape Number

Ya tenemos la silueta discretizada, podemos calcular su Chain Code. Para poder decir que se trata de un código específico de la figura necesitamos que sea invariante a rotaciones y que no importe por cuál punto iniciamos el recorrido.

Chain code: 0 0 0 0 3 0 0 3 2 2 3 2 2 2 1 2 1 1

Difference: 3 0 0 0 3 1 0 3 3 0 1 3 0 0 3 1 3 0

Shape no.: 0 0 0 3 1 0 3 3 0 1 3 0 0 3 1 3 0 3

# Shape Number

Ya tenemos la característica de la figura que vamos a usar para comparar con otras y clasificarla ¿Cómo la clasificamos?

- Grado de similitud  $k$
- Distancia Levenshtein
- Sequence Matcher